

A Survey on Structured Sparse Attention Methods in Transformers

Valerie Salinas

Department of Computer Science, University of Texas at San Antonio

CS 4953-903: High Performance Machine Learning

Sushil K. Prasad

May 4, 2026

TABLE OF CONTENTS

Abstract.....	5
Introduction.....	5
2. Background.....	6
3. Problem Setting.....	7
4. Structured Sparse Attention: Classification Overview.....	8
5. Fixed Structured Sparse Attention.....	9
6. Local Sparse Attention.....	10
6.1 Representative Methods.....	10
6.2 Design Choices and Analysis.....	11
6.3 Performance Trade-Offs.....	12
6.4 Practical Considerations and Comparisons.....	13
7. Hybrid Global–Local Attention.....	13
7.1 Representative Methods.....	14
7.2 Design Choices and Analysis.....	15
7.3 Performance Trade-Offs.....	16
7.4 Practical Considerations and Comparisons.....	16
8. Pattern-Based Sparse Attention.....	17
8.1 Representative Methods.....	18
8.2 Design Choices and Analysis.....	19
8.3 Performance Trade-Offs.....	20
8.4 Practical Considerations and Comparisons.....	20
9. Adaptive / Learnable Sparsity.....	21
9.1 Representative Methods.....	21

9.2 Design Choices and Analysis.....	22
9.3 Performance Trade-Offs.....	23
9.4 Practical Considerations and Comparisons.....	23
10. Memory-Augmented Methods.....	24
10.1 Representative Methods.....	24
10.2 Design Choices and Analysis.....	25
10.3 Performance Trade-Offs.....	26
10.4 Practical Considerations and Comparisons.....	26
11. Comparative Discussion of Approaches.....	27
11.1 Comparison of Methods.....	27
11.2 Analysis of Trade-Offs.....	28
12. Current Trends in Sparse Transformers.....	29
12.1 Long-Context Large Language Models.....	29
12.2 Mixture-of-Experts and Sparse Computation.....	29
12.3 Hardware-Aware Transformer Design.....	30
12.4 Training-Time Sparse Attention.....	30
12.5 Combining Memory and Sparse Attention.....	30
13. Open Problems and Future Directions.....	30
13.1 Learnable Sparsity Patterns.....	31
13.2 Improving Global Reasoning.....	31
13.3 Stability and Training Efficiency.....	31
13.4 Multimodal Sparse Transformers.....	31
13.5 Hardware–Algorithm Co-Design.....	32
14. Conclusion.....	32

References.....	34
Appendices.....	37
A.1 Classification of Structured Sparse Attention Methods.....	37
A.2 Annotated Bibliography.....	38

Abstract

Transformer models have become a dominant architecture for a wide range of machine learning tasks, including natural language processing and computer vision. However, their self-attention mechanism scales quadratically with sequence length, resulting in high computational and memory costs that limit their ability to process long inputs efficiently.

To address this limitation, sparse attention methods restrict each token to attend to only a subset of other tokens, reducing complexity while preserving important dependencies. Structured sparse attention further constrains these interactions using predefined patterns such as sliding windows, block-based connections, and global tokens, enabling efficient long-sequence modeling.

This survey presents a structured overview of sparse attention methods in Transformer architectures. We organize existing approaches(methods) into five categories based on their sparsity patterns: local sparse attention, hybrid global–local attention, pattern-based sparsity, adaptive or learnable sparsity, and memory-augmented methods. For each category, we examine representative foundational, journal, and recent conference works, and analyze their design choices and performance trade-offs.

Our analysis shows that structured sparse attention methods significantly improve scalability, but no single approach fully resolves the trade-off between efficiency and the ability to capture global context. We conclude by discussing current trends and highlighting future directions, including adaptive sparsity mechanisms, integration with memory-based approaches, and improved support for long-context and multimodal applications.

1 INTRODUCTION

Transformer models have become a fundamental architecture in modern machine learning, achieving strong performance across a wide range of tasks such as natural language processing, computer vision, and sequence modeling. Their success is largely due to the self-attention mechanism (Vaswani et al., 2017), which allows each token in a sequence to capture relationships with all other tokens. This ability to model global dependencies has made Transformers more effective than earlier approaches such as recurrent and convolutional neural networks.

Despite their effectiveness, Transformers face a major limitation when applied to long sequences. The self-attention mechanism computes interactions between every pair of tokens, resulting in computational and memory complexity that grows quadratically with sequence length (Vaswani et al., 2017). As input sizes increase, this cost becomes prohibitive, making it difficult to process long documents, extended conversations, or high-resolution data efficiently.

To address this challenge, a large body of work has focused on improving the efficiency of attention mechanisms. One common approach is sparse attention, where each token

attends to only a subset of other tokens instead of the full sequence. By reducing the number of attention connections, sparse methods significantly lower computational cost while still capturing important relationships in the data. (Child et al., 2019).

Within this area, structured sparse attention has emerged as a particularly effective direction. Rather than selecting connections randomly, these methods impose predefined patterns—such as sliding windows, block structures, global tokens, or periodic connections—to guide how tokens interact. These structured designs aim to preserve essential contextual information while enabling scalable computation for long sequences. (Beltagy et al., 2020; Zaheer et al., 2020)

This survey focuses on structured sparse attention methods in Transformer architectures. We organize existing approaches based on the type of sparsity pattern they use, including local sparse attention, hybrid global–local methods, pattern-based sparsity, adaptive or learnable sparsity, and memory-augmented alternatives. For each category, we examine representative foundational, journal, and recent conference works, and analyze how different designs balance efficiency, flexibility, and the ability to capture long-range dependencies.

The main goal of this survey is to provide a unified view of structured sparse attention as a spectrum of design choices rather than a collection of isolated methods. By comparing these approaches, we highlight the trade-offs involved in scaling Transformers to long sequences and identify key challenges that remain in achieving both efficiency and strong global reasoning.

Note: This survey was developed with the assistance of ChatGPT (OpenAI) for drafting, organization, and figure generation. Figures 4 and 5 were generated with the assistance of ChatGPT (OpenAI) and refined by the author.

2 BACKGROUND

Transformer models are built on the idea of attention, a mechanism that allows a model to focus on the most relevant parts of an input sequence when making predictions. Early work in neural machine translation introduced attention as a way to improve sequence-to-sequence models by allowing the decoder to selectively focus on different parts of the input (Bahdanau et al., 2014; Luong et al., 2015). Instead of compressing an entire sequence into a single fixed representation, attention computes a weighted combination of all input tokens, enabling more accurate and flexible modeling of dependencies.

The introduction of the Transformer architecture marked a major shift in how sequence data is processed (Vaswani et al., 2017). Unlike recurrent or convolutional models, Transformers rely entirely on self-attention mechanisms to compute relationships between tokens. This allows the model to process all elements of a sequence in parallel, improving both efficiency and scalability. At the core of this architecture is scaled dot-product attention, which operates using three components: queries, keys, and values. For each token, a query vector is compared with key vectors from all other tokens to determine attention

weights, which are then used to compute a weighted sum of value vectors. This process allows the model to capture both local and global dependencies within the sequence.

However, a key limitation of self-attention is that it computes interactions between every pair of tokens, resulting in an attention matrix of size $n \times n$, which leads to quadratic time and memory complexity (Vaswani et al., 2017). While this design enables rich contextual modeling, it becomes inefficient for long sequences, where both computation and memory requirements grow rapidly.

To address this limitation, researchers have explored a range of approaches for reducing the cost of attention. One line of work focuses on introducing sparsity by limiting the number of attention connections, allowing each token to attend to only a subset of the sequence (Child et al., 2019; Zaheer et al., 2020; Beltagy et al., 2020). These methods use structured patterns such as local windows, global tokens, or block-based connections to reduce computational complexity while still capturing important dependencies.

These foundational developments attention mechanisms, the Transformer architecture, and early sparse attention methods form the basis for modern research on efficient Transformer models. They motivate the need for structured sparse attention, which aims to balance the trade-off between computational efficiency and the ability to model long-range relationships in sequence data.

3 PROBLEM SETTING:

The primary challenge addressed in this survey is the computational and memory cost of the self-attention mechanism in Transformer models when applied to long sequences. In standard self-attention, each token attends to every other token in the sequence. For an input of length n , this results in an attention matrix of size $n \times n$, leading to quadratic time and memory complexity, $O(n^2)$ (Vaswani et al., 2017).

As the sequence length increases, this cost grows rapidly, making it difficult to scale Transformers to long documents, extended conversations, or high-resolution data. In practice, this leads to high memory usage and increased computation time, limiting the effectiveness of standard Transformers in long-context scenarios.

Sparse attention methods aim to reduce this cost by restricting each token to attend to only a subset of other tokens. Instead of computing all pairwise interactions, each token attends to at most k other tokens, where $k \ll n$. This reduces the overall complexity to $O(n \cdot k)$, which is significantly more efficient when k remains small (Child et al., 2019).

However, this improvement introduces a new challenge. By limiting attention connections, the model may lose access to important global context. Long-range dependencies that are naturally captured by full attention can become harder to model under restricted connectivity.

The central problem can therefore be understood as a trade-off between efficiency and expressiveness. Reducing attention connections improves computational efficiency, but

overly restrictive sparsity patterns can limit the model’s ability to capture important relationships across the sequence.

This survey focuses on how structured sparse attention methods address this trade-off. Specifically, it examines how different sparsity patterns are designed to balance efficient computation with the ability to preserve both local and global context in long sequences.

4 STRUCTURED SPARSE ATTENTION: CLASSIFICATION OVERVIEW

As the number of approaches for improving Transformer efficiency has grown (Tay et al., 2020), it has become increasingly difficult to compare methods directly. Different models introduce sparsity in different ways, use different design choices, and target different aspects of the attention mechanism. Without a clear structure, these approaches can appear disconnected, making it challenging to understand how they relate to one another or how they address the core problem of efficient long-sequence modeling.

To address this, a classification framework is needed to organize existing work in a meaningful and consistent way. Rather than grouping methods by application domain or chronology, this survey categorizes approaches based on how attention connections are restricted. This perspective highlights the underlying design choices that define each method and makes it easier to compare their strengths, limitations, and trade-offs.

The key idea of this survey is that structured sparse attention methods can be understood as different ways of limiting attention interactions using predefined or learned patterns. Instead of allowing every token to attend to all others, these methods impose structure on the attention matrix, determining which connections are allowed and which are ignored. This structure can take many forms, including local neighborhoods, global tokens, periodic patterns, or dynamically learned connections.

Based on this perspective, the methods discussed in this survey are organized into five main categories:

- **Local sparse attention**, where tokens attend only to nearby tokens within a fixed window
- **Hybrid global–local attention**, which combines local connections with a small number of global interactions
- **Pattern-based sparse attention**, where fixed deterministic patterns define attention connections
- **Adaptive or learnable sparsity**, where the model dynamically determines which tokens to attend to
- **Memory-augmented methods**, which extend the model’s capacity by introducing additional memory components instead of restricting attention

Each category represents a different point in the design space, reflecting a trade-off between computational efficiency, flexibility, and the ability to capture long-range dependencies. By organizing the literature in this way, this survey provides a unified view of structured sparse attention as a spectrum of design choices rather than a collection of independent techniques.

This five-class taxonomy provides a more effective framework than alternative groupings based on application domain or chronology, because it directly captures the underlying design principle of each method, how attention connections are structured or extended. By organizing methods based on connection patterns rather than implementation details, this classification enables clearer comparison across approaches and highlights the fundamental trade-offs between efficiency, flexibility, and global context. This perspective also makes it easier to identify how newer methods extend or combine existing ideas, reinforcing the view of structured sparse attention as a unified design space.

5 FIXED STRUCTURED SPARSE ATTENTION

Fixed structured sparse attention methods reduce the computational cost of self-attention by imposing predefined patterns on how tokens interact. Instead of allowing each token to attend to all other tokens, these methods restrict attention connections according to a fixed structure that is determined before training. This structure defines which tokens are allowed to exchange information and which interactions are ignored.

The key idea behind fixed sparsity is to reduce the number of attention connections while still preserving the most important relationships in the sequence. In full self-attention, each token attends to all others, resulting in $O(n^2)$ complexity. In contrast, fixed sparse patterns limit each token to a subset of connections, reducing the complexity to approximately $O(n \cdot k)$, where $k \ll n$ (Child et al., 2019). By limiting attention to structured patterns such as local neighborhoods, blocks, or factorized connections, these methods make it possible to efficiently process long sequences.

Unlike adaptive approaches, fixed structured methods do not change their attention patterns based on the input data. This makes them easier to implement, more predictable, and often more efficient in practice, especially for hardware optimization. Because the attention structure is known in advance, these methods can take advantage of parallel computation and optimized memory access patterns.

However, the use of predefined patterns can also introduce limitations. Since attention connections are restricted, important dependencies may be missed if they fall outside the allowed structure. This can reduce the model’s ability to capture long-range relationships, especially in tasks that require global context.

In this section, we examine two major types of fixed structured sparse attention: local sliding window attention and pattern-based (block or factorized) attention. These approaches represent some of the earliest and most widely used methods for improving the

efficiency of Transformer models and form the foundation for many later developments in structured sparse attention.

6 LOCAL SPARSE ATTENTION

We begin with local sparse attention, which represents the most restrictive form of structured sparsity and serves as a baseline for comparison with more expressive methods. Local sparse attention is one of the simplest and most intuitive approaches to reducing the cost of self-attention. In this setting, each token attends only to a limited number of nearby tokens within a predefined window, rather than the entire sequence. This restriction reduces the number of attention interactions from all pairwise connections to a localized subset, significantly improving computational efficiency.

By limiting each token to at most w neighboring connections, where $w \ll n$, the computational complexity is reduced from $O(n^2)$ to approximately $O(n \cdot w)$ (Child et al., 2019). This makes local attention particularly effective for long sequences, especially in settings where nearby elements carry the most relevant information.

Early work such as Sparse Transformer (Child et al., 2019) demonstrated how local attention can be combined with structured sparsity patterns, such as strided attention, to reduce computation while still maintaining reasonable performance. However, purely local attention introduces a key limitation: it cannot directly model long-range dependencies. This limitation is further highlighted in Transformer-XL (Dai et al., 2019), which introduces segment-level recurrence to extend context beyond a fixed local window.

6.1 REPRESENTATIVE METHODS

Sliding window attention is the most common form of local sparsity. In this approach, each token attends only to a fixed window of neighboring tokens, creating a banded structure in the attention matrix.

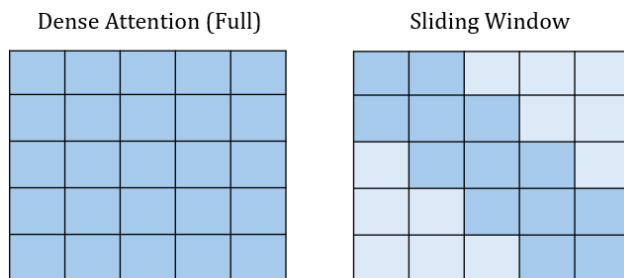


Figure 1: Comparison of Dense and Sliding Window Attention Patterns.

Left: Dense self-attention where each token attends to all other tokens, resulting in a fully populated attention matrix.
Right: Sliding window attention where each token attends only to a fixed number of neighboring tokens, producing a banded diagonal structure.

This structure allows the model to efficiently capture short-range dependencies while avoiding the quadratic cost of full attention. Because the window size is fixed, the computation scales linearly with sequence length.

While this approach is simple, it is highly effective for modeling local relationships in sequential data. However, because each token only has access to nearby information, long-range dependencies must be captured indirectly across multiple layers.

While sliding window attention provides the foundation for local sparsity, several methods extend this idea in different ways to improve performance and applicability.

One important extension is the use of global tokens, as seen in (Beltagy et al., 2020), where a small number of tokens are allowed to attend to the entire sequence. This hybrid design preserves local efficiency while enabling selective access to global context, making it effective for long-document tasks.

Another variation includes dilated sliding window attention, also used in Longformer, where tokens attend to spaced-out positions. This increases the receptive field without significantly increasing computation.

More recent models such as Mistral 7B (Jiang et al., 2023) demonstrate that sliding window attention can be effectively used in modern large language models. Mistral combines sliding-window attention with a rolling buffer cache, allowing information to persist across layers while keeping memory usage low.

In addition, SWAT (Sliding Window Attention Training) (Bai et al., 2024) shows that applying sliding window attention during both training and inference improves stability and efficiency. This highlights an important design choice: models should be trained with the same sparsity pattern used at inference time.

Some approaches also combine local attention with other efficient mechanisms. For example, hybrid designs integrate sliding window attention with linear attention or memory-based methods to balance efficiency and long-range reasoning.

Beyond natural language processing, local attention has also been applied to vision tasks. In these settings, nearby pixels often share strong spatial relationships, making localized attention particularly effective. Neighborhood-based attention methods restrict interactions to spatially close regions, improving efficiency while preserving important structural information (Hassani et al., 2023; Parmar et al., 2018).

6.2 DESIGN CHOICES AND ANALYSIS

Local sparse attention methods differ based on several key design choices that influence both performance and efficiency.

One important factor is the window size, which determines how much local context each token can access. Larger windows allow the model to capture more information, but they also increase computational cost. In contrast, smaller windows are more efficient but may

fail to capture important dependencies beyond the immediate neighborhood (Child et al., 2019).

Another key design choice is whether the attention is causal or bidirectional. In autoregressive language models, causal attention is used so that each token attends only to previous positions. This ensures that predictions are made based only on past information. In contrast, encoder-based models often use bidirectional attention, allowing tokens to attend to both past and future positions within the local window to capture richer contextual relationships.

Local attention can also be extended using dilated windows, where tokens attend to spaced positions rather than strictly adjacent ones. This idea, used in models such as Longformer (Beltagy et al., 2020), increases the effective receptive field without significantly increasing computational cost.

Despite its efficiency, a major limitation of local attention is its inability to capture long-range dependencies directly. Because tokens only interact with nearby neighbors, information must propagate across multiple layers to connect distant positions. To address this limitation, many models incorporate additional mechanisms. For example, Longformer introduces global tokens that attend to the entire sequence (Beltagy et al., 2020), Transformer-XL uses recurrence to extend context across segments (Dai et al., 2019), and more recent models such as Mistral use caching strategies to maintain information over longer contexts (Jiang et al., 2023).

Finally, the alignment between training and inference is an important consideration. Methods such as SWAT (Sliding Window Attention Training) show that applying the same sparse attention pattern during both training and inference improves stability and overall performance (Bai et al., 2024).

These design choices highlight that local sparse attention is not a single fixed method, but rather a flexible framework with multiple trade-offs between efficiency, context coverage, and model complexity.

6.3 PERFORMANCE TRADEOFFS

Local sparse attention offers significant efficiency benefits by reducing the cost of self-attention from quadratic to near-linear complexity, making it well suited for long-sequence tasks (Child et al., 2019). This efficiency has made it a practical choice for large-scale applications, including language modeling, code generation, and vision tasks.

The main advantage of local attention lies in its simplicity and scalability. Because the attention pattern is fixed and structured, it can be implemented efficiently and scaled to long sequences with relatively low memory overhead. This makes it particularly effective in scenarios where local context is sufficient for accurate modeling.

However, these benefits come with important trade-offs. By restricting attention to a limited neighborhood, local attention lacks direct access to global context. As a result, it can struggle with tasks that require long-range reasoning, since information must propagate

indirectly across multiple layers. This can lead to weaker performance compared to full attention in settings where distant dependencies are critical.

To mitigate this limitation, many models extend local attention with additional mechanisms. For example, Longformer incorporates global attention to provide selective access to the full sequence (Beltagy et al., 2020), Transformer-XL introduces recurrence to extend context across segments (Dai et al., 2019), and Mistral uses caching strategies to preserve information over longer contexts (Jiang et al., 2023). Similarly, Sparse Transformer combines local attention with structured sparsity patterns to improve coverage while maintaining efficiency (Child et al., 2019).

These approaches highlight a central trade-off in local sparse attention: while it provides strong efficiency and scalability, it often requires augmentation to achieve robust performance on tasks involving long-range dependencies.

6.4 PRACTICAL CONSIDERATIONS AND COMPARISONS

In practice, local sparse attention is widely used due to its efficiency and simplicity. However, different implementations offer different trade-offs.

Method	Pattern	Global Context	Core Mechanism	Trade-off
Sparse Transformer	Local + Strided	Limited	Structured sparsity	Rigid patterns
Transformer-XL	Local + Memory	Yes	Recurrence across segments	More complexity
Longformer	Sliding Window + Global	Yes	Hybrid local + global	Extra design choices
Mistral	Sliding Window	Limited	Efficient LLM inference	Weak long-range
SWAT	Sliding Window	Limited	Train-time sparsity	Still local

Table 1: Comparison of Local Sparse Attention Methods and Their Trade-offs

From this comparison, several trends can be observed. Sliding window attention provides a strong baseline due to its simplicity and efficiency. However, methods that extend local attention with additional mechanisms, such as global tokens or recurrence, achieve better performance on tasks requiring long-range dependencies.

Compared to hybrid global–local attention, local methods achieve higher efficiency but lack direct mechanisms for global context. In contrast, pattern-based methods provide better long-range connectivity through structured links, while adaptive methods improve flexibility at the cost of higher complexity.

Overall, local sparse attention serves as a fundamental building block for many efficient Transformer architectures. While it is highly effective for capturing local structure, it is often combined with other techniques to overcome its limitations and improve global reasoning.

7 HYBRID GLOBAL–LOCAL ATTENTION

Building on local attention, hybrid methods introduce global connections to address its limitations. Hybrid global–local attention methods aim to balance efficiency and expressiveness by combining local attention patterns with a small number of global connections. Instead of relying on a single sparsity structure, these methods integrate multiple types of attention mechanisms within the same model, allowing them to better approximate full attention while maintaining reduced computational cost.

The key idea is to preserve the efficiency of local attention while introducing mechanisms that enable direct access to long-range dependencies. In these models, most tokens attend only to nearby neighbors, while a small subset of tokens often referred to as global tokens can attend to the entire sequence. This creates a structured attention pattern that maintains near-linear complexity while improving the model’s ability to capture global context.

Early work such as Longformer (Beltagy et al., 2020), ETC (Ainslie et al., 2020), and BigBird (Zaheer et al., 2020) demonstrates that combining local and global attention can achieve strong performance on long-sequence tasks while remaining computationally efficient.

7.1 REPRESENTATIVE METHODS

Several key models illustrate different ways of implementing hybrid global–local attention.

Longformer (Beltagy et al., 2020) combines sliding window attention with task-specific global tokens. Most tokens attend locally, while selected tokens such as classification or query tokens are assigned global attention. This design enables efficient processing of long documents while preserving important global relationships.

BigBird (Zaheer et al., 2020) extends this approach by incorporating random attention connections alongside local and global patterns. These additional connections improve connectivity across the sequence and allow the model to better approximate full attention. Notably, BigBird provides theoretical guarantees that such sparse attention patterns can retain key properties of standard Transformers.

ETC (Encoding Long and Structured Inputs in Transformers) (Ainslie et al., 2020) introduces a two-level structure in which tokens are explicitly divided into local and global roles. Global tokens aggregate and distribute information across the sequence, making this approach particularly effective for structured inputs such as long documents with hierarchical organization.

More recent approaches focus on practical deployment and scalability. LSG Attention (Local–Sparse–Global Attention) extends pretrained Transformer models to longer sequences by combining local, sparse, and global attention patterns within a unified framework. This approach enables efficient scaling to long inputs without requiring full retraining, making it well-suited for real-world applications.

In addition, modern large language models often adopt hybrid attention designs at the architectural level. In these models, most layers apply local attention, while only a subset

incorporate global attention mechanisms. This layer-wise strategy improves computational efficiency while preserving the ability to capture long-range dependencies.

This hybrid attention structure is illustrated in Figure 2. The diagonal band represents local attention, while the highlighted row and column correspond to global token connections that enable full-sequence interaction.

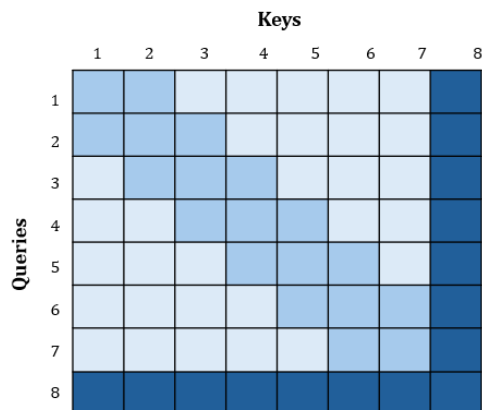


Figure 2: Hybrid Global-Local Attention Pattern.
The diagonal band represents local sliding window attention, while the highlighted row and column indicate a global token that attends to and is attended by all tokens. This structure enables efficient computation while improving long-range information flow.

Some models also include random or structured connections to improve connectivity. For example, BigBird adds random attention links, ensuring that information can propagate efficiently between distant tokens.

This combination creates a sparse but well connected attention graph that approximates full attention without incurring quadratic cost.

Methods	Local	Global	Random	Core Mechanism	Trade-off
Longformer	Yes	Yes	No	Sliding window + task-specific global tokens	Requires token selection
<u>BigBird</u>	Yes	Yes	Yes	Local + global + random attention	More complex pattern
ETC	Yes	Yes	No	Separate global tokens for structure	Higher architectural cost
LSG	Yes	Yes	No	Extend pretrained models to long inputs	Approximation error
Modern LLMs	Yes	Yes	No	Mix local + global across layers	Harder to tune

Table 2: Hybrid Global-Local Attention Methods

7.2 DESIGN CHOICES AND ANALYSIS

Hybrid attention methods differ based on several important design decisions.

One key choice is how global tokens are selected. Some models use task-specific tokens, as in Longformer, while others use fixed global tokens, as in BigBird. Increasing the number of global tokens improves long-range communication but also increases computational cost.

Another important factor is the attention pattern itself. Longformer uses a combination of local and global attention, while BigBird adds random connections. These random links improve connectivity and help information propagate across distant tokens, but they make the model less interpretable.

Hybrid methods also differ in whether they use fixed or adaptive sparsity patterns. Most approaches rely on predefined structures, which are easier to implement and optimize. Adaptive methods offer greater flexibility but are typically harder to train and less stable.

The placement of sparse attention within the architecture is also important. Hybrid attention is most beneficial in encoder settings, where input sequences are long. In contrast, decoder sequences are usually shorter, so the impact is smaller.

Finally, modern models often use layer-level hybrid designs, where local attention is applied in most layers and global attention is used only in selected layers. This approach reduces memory usage while still enabling periodic full-sequence communication.

7.3 PERFORMANCE TRADEOFFS

Hybrid global–local attention provides one of the best trade-offs between efficiency and performance. By combining local and global interactions, these models can capture both short-range and long-range dependencies more effectively than purely local approaches.

The main benefit is that they significantly reduce the computational cost of full attention while maintaining strong performance on long-sequence tasks such as document classification, question answering, and summarization. For example, *BigBird* reduces the quadratic complexity of full attention to linear complexity while still achieving comparable performance.

However, this improved performance comes with increased design complexity. Hybrid models require careful tuning of multiple components, including the number and placement of global tokens, the size of local windows, and the structure of additional connections. These choices can impact both efficiency and accuracy.

7.4 PRACTICAL CONSIDERATIONS AND COMPARISONS

In practice, several factors influence the effectiveness of hybrid attention methods:

Methods	Strength	Limitation	Best Use Case
Longformer	Simple + effective	Needs manual global tokens	Long documents
BigBird	Strong connectivity	Complex design	General long-context tasks
ETC	Handles structure well	More overhead	Structured inputs
LSG	Extends pretrained models	Approximation trade-offs	Efficient scaling
Modern LLMs	Practical + scalable	Hard to tune	Modern LLM systems

Table 3: Comparison of Hybrid Global-Local Attention Methods

From this comparison, several key trends can be observed. Hybrid attention methods improve long-range connectivity by combining local and global interactions, allowing them to capture both short-range and long-range dependencies more effectively than purely local approaches. Simpler methods such as *Longformer* provide efficient solutions for long documents, while more complex approaches like *BigBird* enhance connectivity through additional attention patterns.

Compared to purely local methods, hybrid attention significantly improves long-range reasoning by introducing global tokens. However, unlike adaptive sparsity, these connections are still predefined, limiting flexibility. This positions hybrid methods as a middle ground between efficiency and expressiveness.

Overall, hybrid attention represents a practical balance between efficiency and performance. While these methods significantly improve global reasoning compared to local sparse attention, they introduce additional design complexity and require careful tuning. As a result, hybrid attention has become a widely adopted approach in modern Transformer architectures, particularly for tasks involving long-context modeling.

8 PATTERN-BASED SPARSE ATTENTION

Pattern-based sparse attention defines attention connections using fixed, structured patterns such as blocks or strided layouts. Unlike local attention, which is based on proximity, these methods organize interactions according to predefined structures that determine how tokens can communicate across the sequence.

The key idea is to impose a structured sparsity pattern on the attention matrix so that tokens can interact within specific regions or at regular intervals. This reduces computational cost while still allowing information to propagate across the sequence. In many cases, these patterns create a sparse but well-connected graph, enabling both local and long-range interactions without requiring full attention.

Early work such as *Image Transformer* (Parmar et al., 2018) and *Sparse Transformer* (Child et al., 2019) introduced structured attention layouts, including block and strided patterns, to reduce the quadratic cost of full attention. Later approaches such as *Reformer* (Kitaev et al., 2020) further explored structured sparsity to achieve scalable attention mechanisms for long sequences.

Because the structure is fixed, pattern-based methods are predictable and well-suited for efficient implementation. Their regularity enables optimized memory access and parallel computation, making them attractive for large-scale models and hardware acceleration.

8.1 REPRESENTATIVE METHODS

In contrast to locality-based methods, pattern-based attention defines connectivity through structured layouts. Several methods demonstrate different forms of pattern-based sparsity:

Block attention, introduced in early works such as *Image Transformer* (Parmar et al., 2018) divides the sequence into fixed segments. Tokens attend primarily within their assigned blocks, reducing computation by limiting interactions to smaller regions. This approach is particularly effective when the input has natural structure, such as image patches or document segments.

Strided or factorized attention, used in *Sparse Transformer* (Child et al., 2019), allows tokens to attend to positions at regular intervals across the sequence. These periodic connections introduce long-range interactions while maintaining sparsity, helping information propagate across distant tokens.

Sparse Transformer combines block and strided patterns, demonstrating that carefully designed attention layouts can approximate full attention while significantly reducing computational cost.

Reformer (Kitaev et al., 2020) extends structured sparsity by grouping tokens using hashing techniques and processing them in chunks. While not purely fixed, this approach maintains structured attention patterns and achieves near-linear complexity.

More recent work explores the limitations of fixed patterns. For example, *Routing Transformer* (Roy et al., 2020) introduces content-based grouping, and *Dynamic Sparse Attention* (Liu et al., 2021) shows that fixed patterns may miss important dependencies, motivating more flexible designs.

This structure is illustrated in Figure 3. Diagonal blocks represent local attention within segments, while periodic connections correspond to strided attention across the sequence. The combined pattern integrates both to enable efficient long-range information flow.

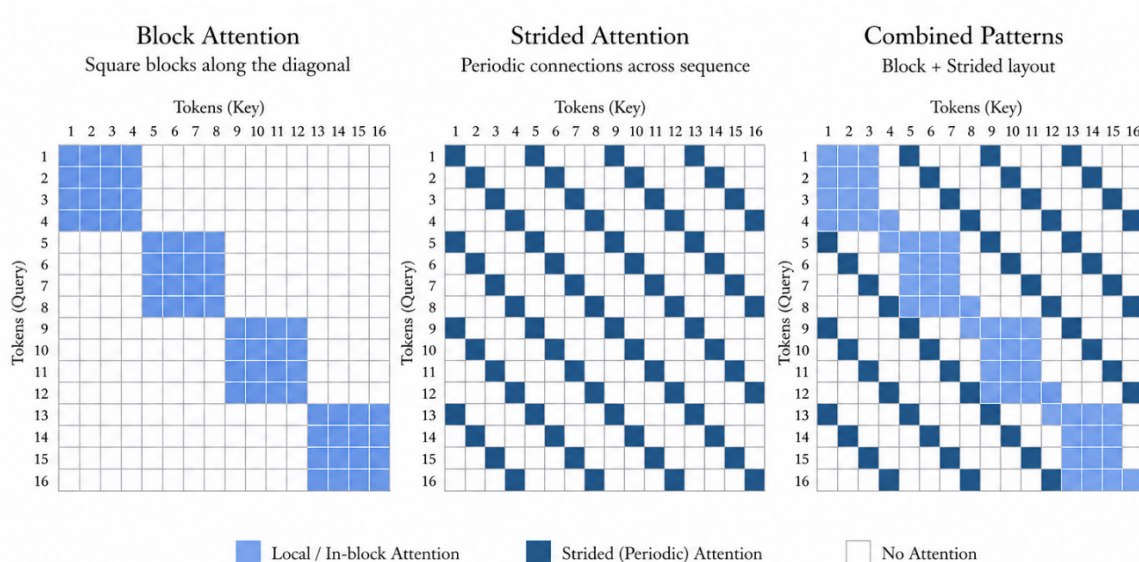


Figure 3: Pattern-Based Attention Structures. The matrices illustrate three types of structured sparse attention patterns. Left: block attention, where tokens attend only within fixed-size blocks along the diagonal. Middle: strided attention, where tokens attend to positions at regular intervals across the sequence. Right: combined block and strided attention, which integrates local block interactions with periodic long-range connections. These structured patterns reduce computational cost compared to full attention while enabling information flow across both local and distant tokens.

8.2 DESIGN CHOICES AND ANALYSIS

Pattern-based sparse attention methods differ based on several key design decisions.

One important choice is the type of pattern used, such as block-based, strided, or factorized layouts. Block patterns are simple and efficient, while strided patterns improve long-range connectivity.

Another factor is the density of the pattern, which determines how many attention connections are allowed. Denser patterns improve accuracy but increase computational cost, while sparser patterns are more efficient but may lose important information.

The coverage of long-range dependencies is also critical. Block attention alone may fail to connect distant tokens, while strided patterns introduce periodic long-range links that improve connectivity.

Pattern-based methods also differ in whether they use strictly fixed patterns or semi-structured approaches. Fixed patterns are easier to implement and optimize, while more flexible designs can better capture complex dependencies but are harder to train.

Finally, hardware efficiency plays a significant role. Regular patterns, such as blocks, are well-suited for GPU acceleration and parallel processing, while irregular patterns are more difficult to optimize.

These design choices highlight a key characteristic of pattern-based attention: it prioritizes structure and efficiency over flexibility.

8.3 PERFORMANCE TRADE-OFFS

Pattern-based sparse attention methods provide a strong balance between scalability and modeling capability. Compared to local attention, they offer greater flexibility by enabling structured long-range interactions. At the same time, their fixed design makes them more efficient and easier to optimize than adaptive approaches.

However, the reliance on predefined patterns introduces limitations. Because the attention structure does not adapt to the input, important dependencies may be missed if they do not align with the chosen pattern. This can reduce performance in tasks where relationships are highly irregular or data-dependent.

Compared to local attention, pattern-based methods provide better long-range connectivity. However, they are less flexible than hybrid or adaptive approaches, which can dynamically adjust attention patterns.

Overall, pattern-based methods highlight an important design trade-off in sparse attention: increasing structure improves efficiency and scalability, but reduces flexibility. These approaches serve as a foundation for more advanced methods that attempt to combine structure with adaptability.

8.4 PRACTICAL CONSIDERATIONS AND COMPARISON

In practice, pattern-based sparse attention is widely used due to its efficiency and hardware-friendly structure. However, different approaches offer different trade-offs.

Method	Pattern	Long-Range Support	Core Mechanism	Trade-off
Image Transformer	Block	Limited	Local blocks for structure	Misses cross block dependencies
Sparse Transformer	Block Strided	Yes	Structured sparse layout	Rigid design
Reformer	Chunked Hashed	Yes	Group tokens for efficiency	Approximate attention
Routing Transformer	Clustered	Yes	Content-based grouping	More complex
Dynamic Sparse Attention	Adaptive Pattern	Yes	Learn Connections dynamically	Harder to Train

Table 4: Comparison of Pattern-Based Sparse Attention Methods

From this comparison, it is clear that purely fixed patterns are efficient but limited in flexibility. Methods that introduce additional structure or partial adaptivity improve performance but increase complexity.

Compared to local sparse attention, pattern-based methods provide improved long-range connectivity through structured connections such as blocks or strided links, rather than relying only on nearby interactions. However, unlike hybrid global–local attention, they do

not include explicit global tokens, which can limit direct access to full-sequence context. In contrast to adaptive sparsity, pattern-based methods maintain fixed and predictable attention structures, making them more efficient and easier to optimize but less flexible in capturing data-dependent relationships. This places pattern-based attention between local and adaptive methods in the design space balancing efficiency with structured long-range connectivity.

9 ADAPTIVE / LEARNABLE SPARSITY

Adaptive or learnable sparsity methods take a different approach to reducing the cost of self-attention by allowing the model to dynamically determine which tokens should attend to each other. Instead of relying on fixed patterns such as local windows or predefined structures, these methods learn attention connections based on the input data.

The key idea is to make sparsity data-dependent. Rather than restricting attention using a fixed layout, the model selectively chooses the most relevant connections for each input. This allows adaptive methods to better capture important relationships, especially in cases where dependencies are irregular or not aligned with predefined patterns.

Early work such as Adaptively Sparse Transformer (Correia et al., 2019) introduced learnable sparse attention distributions using α -entmax, enabling attention weights to become exactly zero. Other approaches, such as Routing Transformer (Roy et al., 2020), further developed this idea by dynamically grouping tokens based on content. These methods demonstrate that attention patterns can be learned rather than manually designed.

While adaptive sparsity improves flexibility and expressiveness, it also introduces additional complexity, as the model must learn both the attention values and the structure of the attention pattern.

9.1 REPRESENTATIVE METHODS

Several approaches illustrate different ways of learning sparse attention patterns.

Adaptive attention span (Sukhbaatar et al., 2019) allows each attention head to learn how far it needs to attend. Instead of using a fixed window size, the model dynamically adjusts its attention range. Some heads focus on local context, while others capture long-range dependencies, improving efficiency without sacrificing performance.

Adaptively Sparse Transformer (Correia et al., 2019) introduces α -entmax, which replaces softmax with a function that can produce sparse probability distributions. This allows the model to learn which attention connections are important while setting others exactly to zero.

Routing Transformer (Roy et al., 2020) uses content-based clustering to group tokens. Attention is computed only within these clusters, allowing the model to focus on relevant subsets of the sequence. This approach enables efficient modeling of long-range dependencies by grouping semantically similar tokens.

More recent work further extends these ideas. Dynamic Sparse Attention (Liu et al., 2021) learns attention masks based on the input, avoiding fixed sparsity patterns. Newer approaches such as mixture-based or routing-based attention mechanisms dynamically select different sparsity patterns for different inputs, improving flexibility and performance.

This distinction is illustrated in Figure 4, where fixed sparsity follows predefined patterns, while adaptive sparsity dynamically selects attention connections based on the input.

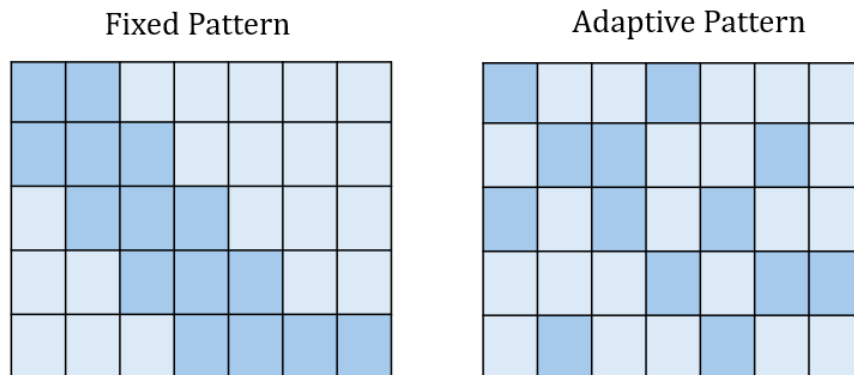


Figure 4: Fixed vs Adaptive Sparse Attention.

Fixed sparsity uses predefined attention structures that remain the same across all inputs, while adaptive sparsity dynamically selects attention connections based on the input data. This allows adaptive methods to capture more flexible and task-dependent relationships compared to fixed patterns.

9.2 DESIGN CHOICES AND ANALYSIS

Adaptive sparse attention methods differ based on several key design decisions.

One important factor is how sparsity is learned. Some methods use continuous functions such as α -entmax to produce sparse attention weights, while others use clustering or routing mechanisms to explicitly select connections.

Another key design choice is the granularity of sparsity. Sparsity can be learned at different levels, including token-level (which tokens attend to each other), head-level (different heads learn different patterns), or layer-level (different layers use different sparsity strategies).

Adaptive methods also vary in their degree of dynamism. Some approaches partially adapt sparsity while maintaining structure, while others fully determine attention patterns based on the input. Fully dynamic approaches are more flexible but are typically harder to train.

A major challenge in adaptive sparsity is training stability. Learning attention patterns introduces additional complexity, and routing or clustering operations can make optimization more difficult. Some methods also introduce extra computational overhead due to the need to compute routing decisions.

Finally, there is a trade-off between flexibility and efficiency. Adaptive methods improve the ability to capture complex dependencies, but they often sacrifice some of the simplicity and hardware efficiency of fixed sparse patterns.

9.3 PERFORMANCE AND TRADE-OFFS

Adaptive and learnable sparsity methods offer greater flexibility than fixed structured approaches. By allowing attention patterns to depend on the input, these models can better capture complex and irregular relationships in the data. This makes them especially useful for tasks where dependencies are not strictly local or aligned with predefined patterns.

However, this flexibility comes at a cost. Learning attention patterns introduces additional computational overhead and increases the complexity of training. These methods can be harder to optimize and often require more careful tuning compared to fixed sparse attention models (Correia et al., 2019; Roy et al., 2020).

Compared to pattern-based and local methods, adaptive sparsity provides improved expressiveness but reduced simplicity. As a result, it represents a fundamental trade-off between modeling power and computational efficiency. Recent work shows that while adaptive methods can better capture long-range and data-dependent relationships, they often sacrifice the hardware efficiency and stability of structured sparse approaches (Liu et al., 2021).

9.4 PRACTICAL CONSIDERATIONS AND COMPARISON

In practice, adaptive sparse attention is powerful but more challenging to implement compared to fixed sparse methods.

Methods	Approach	Core Mechanism	Strength	Limitation
Adaptive Span	Learnable window	Adjust attention range per head	Efficient + flexible	Limited Structure
Adaptively Sparse Transformer	Entmax	Learn sparse attention weights	Interpretable	Harder Optimization
Routing Transformer	Clustering	Group tokens by content	Captures long range	Complex routing
Dynamic Sparse Attention	Learned masks	Input-dependent sparsity	Highly Flexible	Training instability

Table 5: Comparison of Adaptive Sparse Attention Methods

From this comparison, adaptive methods improve flexibility and performance, but at the cost of increased complexity. While they are not as simple or hardware-friendly as fixed sparse approaches, they are better suited for tasks requiring dynamic and context-dependent reasoning.

Compared to fixed structured methods, adaptive sparsity offers greater flexibility by learning attention patterns from data. However, this comes at the cost of increased training complexity and reduced hardware efficiency, making it less practical than local or pattern-based approaches in large-scale systems.

10 MEMORY-AUGMENTED METHODS

Memory-augmented methods take a different approach to improving the scalability of Transformer models. Instead of restricting attention connections to reduce computational cost, these methods introduce additional memory components that allow the model to store and reuse information beyond the current input sequence.

The key idea is to extend the model’s effective context by incorporating memory representations that persist across segments or time steps. Rather than forcing all information to flow through direct token-to-token interactions, memory mechanisms provide an intermediate storage that can be accessed when needed. This allows the model to capture long-range dependencies without requiring full attention across the entire sequence.

Early work such as *Transformer-XL* (Dai et al., 2019) introduced segment-level recurrence, enabling hidden states from previous segments to be reused as memory. This idea was further extended in *Compressive Transformer* (Rae et al., 2019), which stores long-term information in compressed form. More recent approaches, such as *Memorizing Transformers* (Wu et al., 2022), use external memory systems to retrieve past information dynamically.

These methods demonstrate that improving long-context modeling can be achieved not only by reducing attention connections, but also by enhancing the model’s ability to store and reuse information.

10.1 REPRESENTATIVE METHODS

Memory-augmented methods differ from standard self-attention by extending the model’s effective context beyond the current input sequence. While traditional attention mechanisms operate within a fixed window, memory-based approaches incorporate representations from previous segments, enabling information to persist and be reused over time. This allows for improved modeling of long-range dependencies, as illustrated in Figure 5.

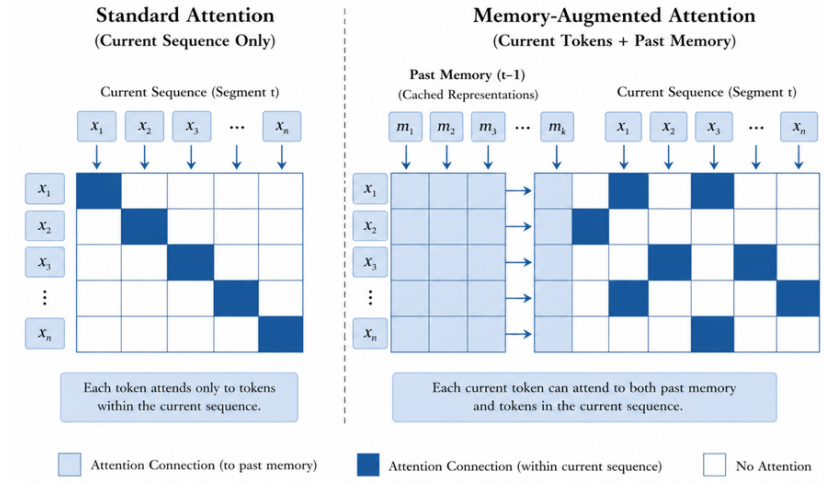


Figure 5: Memory-Augmented Attention. Standard attention (left) operates only on the current input sequence, limiting the model’s context to a fixed window. Memory-augmented attention (right) incorporates representations from previous segments (memory), allowing past information to be stored and reused across segments.

Several approaches extend the basic idea of memory-augmented attention by introducing different mechanisms for storing and retrieving past information. Transformer-XL leverages recurrence to reuse hidden states across segments, effectively increasing the model’s context length (Dai et al., 2019). Compressive Transformer builds on this by introducing compressed memory, enabling long-term storage without linear growth in memory requirements (Rae et al., 2019).

Alternative designs focus on structured or external memory. Memory Transformer uses dedicated memory tokens to provide shared storage within the model (Burtsev et al., 2020), while Memorizing Transformers employ external key–value memory with retrieval based on similarity search (Wu et al., 2022). More recent work combines these ideas, integrating recurrence and memory tokens to further extend context across segments (Bulatov et al., 2022).

10.2 DESIGN CHOICES AND ANALYSIS

Memory-augmented methods differ based on several important design decisions.

One key factor is the type of memory used. Some models use recurrent memory (Transformer-XL), while others use compressed memory (Compressive Transformer), explicit memory tokens (Memory Transformer), or external memory systems (Memorizing Transformers). Each approach offers different trade-offs between efficiency and expressiveness.

Another important design choice is the size and capacity of memory. Larger memory allows the model to retain more context, but increases storage and retrieval costs. Methods such as Compressive Transformer address this by compressing older information.

The mechanism for reading and writing memory is also critical. Some models store all past information, while others selectively update memory based on importance. Efficient memory management is necessary to prevent redundancy and maintain useful representations.

Memory persistence is another key consideration. Some methods maintain short-term memory within a limited window, while others support long-term storage across many segments. External memory approaches enable persistent knowledge beyond the current input.

Finally, memory must be carefully integrated with attention mechanisms. Memory can be treated as additional tokens, accessed through retrieval, or incorporated as a separate module. These choices affect both performance and implementation complexity.

10.3 PERFORMANCE TRADE-OFFS

Memory-augmented methods provide strong capabilities for modeling long sequences by allowing the model to retain and reuse past information. This makes them particularly effective for tasks involving long-term dependencies, such as document understanding and long-form text generation.

The main advantage of these methods is their ability to extend context beyond the limits of standard attention. For example, Transformer-XL improves language modeling performance by reusing past hidden states, while Memorizing Transformers benefit from large external memory that grows over time.

However, these benefits come with increased complexity. Memory mechanisms introduce additional parameters and require careful design to manage storage, retrieval, and updates. External memory systems may also introduce latency due to retrieval operations.

Compared to sparse attention methods, memory-based approaches offer stronger long-range modeling but are generally more complex and less efficient to implement. As a result, they represent a trade-off between long-term context and system simplicity.

10.5 PRACTICAL CONSIDERATIONS AND COMPARISONS

In practice, memory-augmented methods are powerful but require careful design to balance performance and efficiency.

Methods	Memory Type	Core Mechanism	Strength	Limitation
Transformer-XL	Recurrent	Cache past hidden states	Extends context efficiently	Limited memory size
Compressive Transformer	Compressed memory	Store long-term info compactly	Longer context	Information Loss
Memory Transformer	Memory Tokens	Shared global storage	Structured memory access	Extra parameters
Memorizing Transformer	External Memory	kNN retrieval of past data	Very large context	Retrieval Cost
Recurrent Memory Transformer	Hybrid Memory	Combine recurrence + memory tokens	Strong long-range modeling	Higher complexity

Table 6: Comparison of Memory-Augmented Transformer Methods

From this comparison, it is clear that memory-based methods improve long-range dependency modeling but introduce additional complexity. Simpler approaches such as Transformer-XL are easier to implement, while more advanced methods offer greater context at the cost of increased computation.

11 COMPARATIVE DISCUSSION OF APPROACHES

The methods discussed in this survey represent different strategies for improving the efficiency of Transformer models when processing long sequences. While each approach reduces computational cost in a different way, they all introduce trade-offs between efficiency, flexibility, and the ability to capture long-range dependencies.

To better understand these differences, we compare the main categories of structured sparse attention based on key design factors, including computational efficiency, ability to model long-range context, flexibility of the attention structure, and overall implementation complexity.

11.1 COMPARISON OF METHODS

To bring together the approaches discussed throughout this survey, it is useful to compare their key characteristics across several dimensions. Table 10.1 summarizes the major attention mechanisms, evaluating them in terms of efficiency, ability to capture long-range context, flexibility, and overall complexity. This comparison provides a unified view of the trade-offs involved in designing scalable Transformer architectures.

Method	Efficiency	Long-Range Context	Flexibility	Complexity
Local sparse Attention	High	Weak	Low	Low
Hybrid Global-Local Attention	Medium	Strong	Medium	Medium
Pattern Based Sparse Attention	High	Medium	Low	Low
Adaptive/Learnable Sparsity	Medium	Strong	High	High
Memory-Augmented Methods	Medium	Strong	Medium	High

Table 7: Comparative Analysis of Efficient Attention Methods

From this comparison, several important trends can be observed. Local and pattern-based sparse attention methods offer high efficiency and simplicity, but are limited in their ability to model long-range dependencies. Hybrid approaches improve long-range context by combining local and global interactions, achieving a balance between efficiency and performance. Adaptive sparsity methods provide the highest flexibility by learning input-dependent attention patterns, but this comes at the cost of increased complexity and reduced efficiency. In contrast, memory-augmented methods extend context through stored representations, enabling strong long-range modeling while introducing additional computational overhead.

These differences suggest that efficient attention mechanisms can be understood along two key dimensions: how they reduce computational cost and how they preserve contextual information. Sparse attention methods primarily focus on limiting connections, while memory-based approaches extend context through storage and retrieval. Hybrid and adaptive methods attempt to bridge these strategies, balancing efficiency with the ability to capture complex, long-range dependencies.

11.2 ANALYSIS OF TRADE-OFFS

Each category of structured sparse attention represents a different design strategy for balancing efficiency and modeling capability.

Local sparse attention methods provide the highest efficiency by restricting interactions to nearby tokens. This makes them well-suited for long sequences where local context is sufficient. However, their inability to directly model long-range dependencies limits their effectiveness in tasks that require global reasoning.

Hybrid global–local attention methods improve upon this limitation by combining local interactions with global connections. This allows them to capture both short-range and long-range dependencies more effectively, resulting in strong overall performance. However, this added capability introduces additional design complexity and requires careful tuning of attention patterns.

Pattern-based sparse attention methods offer a balance between efficiency and structured connectivity. By using predefined patterns such as blocks or strided connections, these approaches enable both local and long-range interactions while maintaining predictable computation. However, their fixed structure limits flexibility, as they cannot adapt to the specific relationships present in the data.

Adaptive or learnable sparsity methods provide the greatest flexibility by allowing the model to determine attention patterns dynamically. This enables them to capture complex and irregular dependencies that fixed patterns may miss. However, this flexibility comes at the cost of increased computational overhead and training complexity, making these methods more difficult to optimize in practice.

Memory-augmented methods take a different approach by improving the model’s ability to store and reuse information rather than restricting attention. These methods are effective for capturing long-range dependencies, especially in very long sequences. However, they introduce additional architectural complexity and require careful design to manage memory effectively.

These comparisons suggest that structured sparse attention methods can be understood along a continuous spectrum, where models trade off between efficiency, flexibility, and global context. Rather than representing separate solutions, these approaches reflect different design choices within a shared framework for scaling attention.

12 CURRENT TRENDS IN SPARSE TRANSFORMERS

Recent advances in Transformer architectures have focused on improving scalability, efficiency, and the ability to handle increasingly long and complex inputs. As models continue to grow in size and are applied to more demanding tasks, several key trends have emerged in the development of structured sparse attention methods (Tay et al., 2020).

12.1 LONG-CONTEXT LARGE LANGUAGE MODELS

A major trend is the development of long-context large language models (LLMs), designed to process extremely long sequences. Full attention becomes impractical at this scale due to its quadratic complexity (Vaswani et al., 2017).

To address this, methods such as sliding window attention, hybrid global–local attention, and memory-based approaches are used to extend context while maintaining efficiency. These designs enable models to process long documents, conversations, and multimodal inputs within computational limits.

12.2 MIXTURE-OF-EXPERTS AND SPARSE COMPUTATION

Another trend is the use of sparsity at the model level through Mixture-of-Experts (MoE) architectures. These models route tokens to a small subset of specialized components, reducing computation while scaling model size (Fedus et al., 2021; Du et al., 2022).

Although MoE does not directly modify attention patterns, it follows the same principle as sparse attention: limiting unnecessary computation. In practice, MoE is often combined with sparse attention to improve scalability.

12.3 HARDWARE-AWARE TRANSFORMER DESIGN

As Transformer models grow larger, hardware efficiency has become a key consideration. Recent work focuses on designing attention mechanisms that align well with modern hardware, such as GPUs and specialized accelerators. Structured sparsity patterns, such as block and local attention, are particularly well-suited for hardware optimization because they allow for predictable memory access and parallel computation.

In addition, techniques such as optimized attention kernels and memory-efficient implementations improve performance without changing the underlying attention structure (Dao et al., 2022). This trend emphasizes that both algorithm design and hardware considerations are important for achieving efficient large-scale models.

12.4 TRAINING-TIME SPARSE ATTENTION

Early sparse attention methods were often applied only during inference, while training still relied on full attention. However, recent approaches extend sparsity to the training process itself. By using sparse attention during training, models can reduce memory usage and computational cost from the beginning, enabling the training of larger models or longer sequences (Fu et al., 2025).

This shift toward training-time sparsity reflects a broader trend of integrating efficiency into the entire model lifecycle, rather than treating it as a post-processing optimization.

12.5 COMBINING MEMORY AND SPARSE ATTENTION

Another emerging direction is the integration of memory mechanisms with sparse attention. Instead of relying solely on restricted attention patterns, these approaches introduce memory tokens or external memory structures to store and retrieve important information.

By combining memory-based methods with structured sparsity, models can maintain global context while still benefiting from reduced computational cost. This hybrid approach addresses one of the main limitations of sparse attention loss of long-range dependencies and represents a promising direction for future research (Dai et al., 2019; Wu et al., 2022).

13 OPEN PROBLEMS & FUTURE DIRECTIONS

Despite significant progress in structured sparse attention, several challenges remain in designing Transformer models that are both efficient and capable of capturing complex long-range dependencies. These open problems highlight important directions for future research.

13.1 LEARNABLE SPARSITY PATTERNS

Many existing sparse attention methods rely on fixed, predefined patterns such as sliding windows or block structures (Child et al., 2019; Beltagy et al., 2020). While these approaches are efficient, they may not adapt well to different types of data or tasks. A key direction for future work is the development of fully learnable sparsity patterns, where the model dynamically determines which attention connections are most important.

Adaptive methods already take steps in this direction, such as learning sparse attention distributions or routing tokens based on content (Correia et al., 2019; Roy et al., 2020). However, these approaches often introduce additional training complexity and instability. Future research will focus on designing models that can learn efficient attention structures while remaining stable, interpretable, and scalable.

13.2 IMPROVING GLOBAL REASONING

One of the main limitations of sparse attention is the potential loss of global context. When attention connections are restricted, it can become more difficult for the model to capture long-range dependencies, especially in tasks that require reasoning across distant parts of a sequence (Zaheer et al., 2020).

Future work is likely to explore new hybrid designs that better preserve global information, such as improved combinations of local and global attention (Beltagy et al., 2020), more effective use of global tokens, or integration with memory-based approaches (Dai et al., 2019; Wu et al., 2022). Developing methods that maintain strong global reasoning while reducing computational cost remains a central challenge.

13.3 STABILITY AND TRAINING EFFICIENCY

Some sparse and adaptive attention methods introduce additional complexity during training, which can lead to instability or slower convergence. Learning which tokens to attend to can make optimization more difficult, especially in large-scale models (Correia et al., 2019; Roy et al., 2020).

Improving training stability is an important direction for future work. This may include better initialization strategies, improved normalization techniques, or more robust optimization methods that allow sparse models to train as reliably as standard Transformers.

13.4 MULTIMODAL SPARSE TRANSFORMERS

Transformer models are increasingly being applied to multimodal tasks that combine text, images, audio, and other data types. These tasks often involve very large inputs and complex relationships between modalities, making efficient attention mechanisms even more important.

Early work on structured attention in vision and multimodal settings demonstrates the importance of locality and structured interactions (Parmar et al., 2018). Future research will

explore how structured sparse attention can be adapted for multimodal settings. This includes designing sparsity patterns that capture both local structure within each modality and global interactions across modalities, while maintaining efficiency.

13.5 HARDWARE–ALGORITHM CO-DESIGN

As models continue to scale, it is becoming clear that efficient Transformer design requires coordination between algorithms and hardware. Structured sparsity patterns are often chosen not only for their theoretical efficiency but also for how well they map to modern hardware architectures.

Recent work on optimized attention implementations highlights the importance of hardware-aware design (Dao et al., 2022). Future work will likely focus on hardware algorithm co-design, where attention mechanisms are developed alongside specialized hardware or optimized kernels. This includes designing sparsity patterns that enable efficient parallelization, reduce memory access costs, and take advantage of emerging accelerator technologies.

These open problems suggest that the future of sparse Transformers lies in combining multiple approaches learnable sparsity, memory mechanisms, and hardware-aware design to achieve both efficiency and strong performance. Addressing these challenges will be key to enabling Transformer models to scale to increasingly complex and long-context applications.

14 CONCLUSION

Transformer models are widely used in modern machine learning, but they can become very expensive when working with long sequences. This is because self-attention requires every token to interact with every other token, which leads to high computational and memory costs. As the input gets longer, this becomes harder to manage, especially for tasks like long documents or large datasets.

Structured sparse attention methods help solve this problem by reducing the number of attention connections or by improving how information is stored and reused. Instead of connecting everything to everything, these methods use specific patterns or learned strategies to decide which connections are most important. This makes the model more efficient while still keeping useful information.

In this survey, structured sparse attention methods were grouped into several categories: local attention, hybrid global–local attention, pattern-based methods, adaptive methods, and memory-augmented approaches. This shows that these techniques are different ways of solving the same problem—making attention more efficient.

From this, it becomes clear that no single method is perfect. Methods that focus on efficiency may lose important long-range information, while methods that improve accuracy

can be more complex and harder to run. Memory-based methods offer another option by storing and reusing past information instead of limiting connections.

Overall, structured sparse attention shows that Transformer models can handle longer inputs more efficiently. However, there is still a challenge in balancing efficiency, flexibility, and the ability to capture long-range relationships. Future work will likely combine different ideas, such as adaptive sparsity, memory, and better hardware design, to improve performance even further.

REFERENCES

- [1] Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. 2014. Neural Machine Translation by Jointly Learning to Align and Translate. arXiv preprint arXiv:1409.0473. <https://arxiv.org/abs/1409.0473>
- [2] Luong, Minh-Thang, Hieu Pham, and Christopher D. Manning. 2015. Effective Approaches to Attention-based Neural Machine Translation. arXiv preprint arXiv:1508.04025. <https://arxiv.org/abs/1508.04025>
- [3] Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In Proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS). <https://arxiv.org/abs/1706.03762>
- [4] Child, Rewon, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating Long Sequences with Sparse Transformers. arXiv preprint arXiv:1904.10509. <https://arxiv.org/abs/1904.10509>
- [5] Tay, Yi, Mostafa Dehghani, Dara Bahri, and Donald Metzler. 2020. Efficient Transformers: A Survey. arXiv preprint arXiv:2009.06732. <https://arxiv.org/abs/2009.06732>
- [6] Beltagy, Iz, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. arXiv preprint arXiv:2004.05150. <https://arxiv.org/abs/2004.05150>
- [7] Nawrot, Piotr, Robert Li, Renjie Huang, Sebastian Ruder, Kelly Marchisio, and Edoardo M. Ponti. 2026. The Sparse Frontier: Sparse Attention Trade-offs in Transformer LLMs. arXiv preprint arXiv:2504.17768. <https://arxiv.org/abs/2504.17768>
- [8] Dao, Tri, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv preprint arXiv:2205.14135. <https://arxiv.org/abs/2205.14135>
- [9] Phuong, Mary, and Marcus Hutter. 2022. Formal Algorithms for Transformers. arXiv preprint arXiv:2207.09238. <https://arxiv.org/abs/2207.09238>
- [10] He, Bobby, and Thomas Hofmann. 2024. Simplifying Transformer Blocks. arXiv preprint arXiv:2311.01906. <https://arxiv.org/abs/2311.01906>
- [11] Zaheer, Manzil, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big Bird: Transformers for Longer Sequences. arXiv preprint arXiv:2007.14062. <https://arxiv.org/abs/2007.14062>
- [12] Roy, Aurko, Mohammad Saffar, Ashish Vaswani, and David Grangier. 2021. Efficient Content-Based Sparse Attention with Routing Transformers. arXiv preprint arXiv:2003.05997. <https://arxiv.org/abs/2003.05997>

- [13] Ainslie, Joshua, Santiago Ontanon, Chris Alberti, Vaclav Cvicek, Zachary Fisher, Philip Pham, Anirudh Ravula, Sumit Sanghai, Qifan Wang, and Li Yang. 2020. ETC: Encoding Long and Structured Inputs in Transformers. In Proceedings of EMNLP. <https://arxiv.org/abs/2004.08483>
- [14] Fedus, William, Barret Zoph, and Noam Shazeer. 2021. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. arXiv preprint arXiv:2101.03961. <https://arxiv.org/abs/2101.03961>
- [15] Du, Nan, Yanping Huang, Andrew M. Dai, Zhifeng Chen, Orhan Firat, Noah Constant, Yonghui Wu, et al. 2022. GLaM: Efficient Scaling of Language Models with Mixture-of-Experts. arXiv preprint arXiv:2112.06905. <https://arxiv.org/abs/2112.06905>
- [16] Hassani, Ali, Steven Walton, Jiachen Li, Shen Li, and Humphrey Shi. 2023. Neighborhood Attention Transformer. arXiv preprint arXiv:2204.07143. <https://arxiv.org/abs/2204.07143>
- [17] Fu, Zichuan, Wentao Song, Yejing Wang, Xian Wu, Yefeng Zheng, Yingying Zhang, Derong Xu, Xuetao Wei, Tong Xu, and Xiangyu Zhao. 2025. Sliding Window Attention Training for Efficient Large Language Models. arXiv preprint arXiv:2502.18845. <https://arxiv.org/abs/2502.18845>
- [18] Liu, Dong, and Yanxuan Yu. 2025. π -Attention: Periodic Sparse Transformers for Efficient Long-Context Modeling. arXiv preprint arXiv:2511.10696. <https://arxiv.org/abs/2511.10696>
- [19] Sukhbaatar, Sainbayar, Edouard Grave, Piotr Bojanowski, and Armand Joulin. 2019. Adaptive Attention Span in Transformers. arXiv preprint arXiv:1905.07799. <https://arxiv.org/abs/1905.07799>
- [20] Burtsev, Mikhail S., Yuri Kuratov, Anton Peganov, and Grigory V. Sapunov. 2021. Memory Transformer. arXiv preprint arXiv:2006.11527.
- [21] Dai, Zihang, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. 2019. Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. arXiv preprint arXiv:1901.02860. <https://arxiv.org/abs/1901.02860>
- [22] Rae, Jack W., Anna Potapenko, Siddhant M. Jayakumar, and Timothy Lillicrap. 2019. Compressive Transformers for Long-Range Sequence Modelling. arXiv preprint arXiv:1911.05507. <https://arxiv.org/abs/1911.05507>
- [23] Wu, Yuhuai, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. 2022. Memorizing Transformers. arXiv preprint arXiv:2203.08913. <https://arxiv.org/abs/2203.08913>
- [24] Parmar, Niki, Ashish Vaswani, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2018. Image Transformer. arXiv preprint arXiv:1802.05751. <https://arxiv.org/abs/1802.05751>
- [25] Jiang, Albert Q., Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra

- Singh Chaplot, Diego de las Casas, Florian Bressand, et al. 2023. Mistral 7B. arXiv preprint arXiv:2310.06825. <https://arxiv.org/abs/2310.06825>
- [26] Liu, Zhuang, et al. 2021. Transformer Acceleration with Dynamic Sparse Attention. ArXiv preprint arXiv:2110.11299. <https://arxiv.org/abs/2110.11299>

APPENDIX

A.1 Classification of Structured Sparse Attention Methods

A.1.1 Foundational Attention Mechanisms

A.1.2 Fixed Structured Sparse Attention

A.1.2.1 Local Sparse Attention

A.1.2.2 Pattern-Based Sparse Attention

A.1.3 Hybrid Global–Local Attention

A.1.4 Adaptive / Learnable Sparsity

A.1.5 Memory-Augmented Methods

A.1.6 Complementary Efficiency Techniques

A.2 Annotated Bibliography

A.2.1 Foundational Attention Mechanisms

[1] Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. 2014. Neural Machine Translation by Jointly Learning to Align and Translate. arXiv preprint arXiv:1409.0473. <https://arxiv.org/abs/1409.0473>

Bahdanau et al. introduce one of the earliest forms of neural attention for sequence-to-sequence models. Their method allows the decoder to focus on different parts of the input sequence instead of relying on one fixed-length representation. This paper is important because it establishes the idea that not all tokens contribute equally at every step. Compared to later Transformer work, this attention mechanism is still tied to recurrent models, but it provides the foundation for self-attention. This connects directly to Luong et al. and Vaswani et al., who further develop attention into more scalable architectures.

[2] Luong, Minh-Thang, Hieu Pham, and Christopher D. Manning. 2015. Effective Approaches to Attention-based Neural Machine Translation. arXiv preprint arXiv:1508.04025. <https://arxiv.org/abs/1508.04025>

Luong et al. extend early attention research by comparing global and local attention mechanisms for neural machine translation. The local attention idea is especially relevant to sparse attention because it limits the range of tokens the model attends to. Compared to Bahdanau et al., this paper gives a clearer distinction between attending broadly and attending selectively. This makes it an important precursor to later sliding-window methods such as Longformer and SWAT. However, because it is still based on recurrent models, it does not solve the scalability issues later addressed by Transformers.

[3] Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In Proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS). <https://arxiv.org/abs/1706.03762>

Vaswani et al. introduce the Transformer architecture, which replaces recurrence with self-attention. This allows tokens to interact directly with all other tokens and enables much greater parallelism than earlier RNN-based attention models. Compared to Bahdanau et al. and Luong et al., this paper makes attention the central mechanism rather than an added component. However, full self-attention has quadratic cost with sequence length, which

becomes the main problem for long inputs. This limitation directly motivates Sparse Transformer, Longformer, BigBird, and many other efficient attention methods.

[4] Phuong, Mary, and Marcus Hutter. 2022. Formal Algorithms for Transformers. arXiv preprint arXiv:2207.09238. <https://arxiv.org/abs/2207.09238>

Phuong and Hutter provide a formal explanation of Transformer algorithms and computations. This paper is useful because it clarifies how Transformer components such as attention, feed-forward layers, and positional encodings operate. Compared to papers like Longformer or BigBird, it does not propose a new sparse attention method. Instead, it supports the background understanding needed to compare different attention designs. This makes it useful for explaining the baseline Transformer structure before discussing sparse or memory-augmented extensions.

A.2.2 Fixed Structured Sparse Attention

A.2.2.1 Local Sparse Attention

[5] Beltagy, Iz, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. arXiv preprint arXiv:2004.05150. <https://arxiv.org/abs/2004.05150>

Longformer introduces a sparse attention pattern designed for long documents. It combines sliding-window local attention with selected global tokens, allowing the model to reduce attention cost while still preserving some long-range context. Compared to Sparse Transformer, Longformer is more directly designed for document-level NLP tasks. Compared to BigBird, it uses a simpler structure because it does not include random attention links. This makes Longformer a strong example of the trade-off between efficiency, simplicity, and limited global reasoning.

[6] Hassani, Ali, Steven Walton, Jiachen Li, Shen Li, and Humphrey Shi. 2023. Neighborhood Attention Transformer. arXiv preprint arXiv:2204.07143. <https://arxiv.org/abs/2204.07143>

Neighborhood Attention Transformer applies local attention to vision tasks by restricting attention to nearby spatial neighborhoods. Compared to Longformer, which applies local attention to text, NAT shows that locality is also useful for images. This supports the idea that structured sparse attention can generalize across domains. Compared to global attention in standard Vision Transformers, neighborhood attention is more efficient and

better aligned with image structure. This paper connects local sparse attention with multimodal and vision-based Transformer applications.

[7] Fu, Zichuan, Wentao Song, Yejing Wang, Xian Wu, Yefeng Zheng, Yingying Zhang, Derong Xu, Xuetao Wei, Tong Xu, and Xiangyu Zhao. 2025. Sliding Window Attention Training for Efficient Large Language Models. arXiv preprint arXiv:2502.18845. <https://arxiv.org/abs/2502.18845>

Fu et al. propose using sliding-window attention during training, not only during inference. This is important because many sparse attention methods focus on making inference cheaper while training still remains expensive. Compared to Longformer, SWAT extends the same local attention idea into the full training process. Compared to hybrid approaches like BigBird, it is simpler but less focused on global context. This paper supports the trend that sparse attention should be integrated throughout the model lifecycle.

[8] Jiang, Albert Q., Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, et al. 2023. Mistral 7B. arXiv preprint arXiv:2310.06825. <https://arxiv.org/abs/2310.06825>

Mistral 7B uses sliding-window attention to improve efficiency in a modern large language model. Compared to Longformer, which targets long-document encoding, Mistral applies local attention in a decoder-style LLM setting. This shows that sliding-window attention remains practical in newer model architectures. Compared to memory-based methods, Mistral’s approach is simpler because it relies on local attention and caching rather than explicit memory modules. It is useful for showing how local sparse attention has influenced current large-scale model design.

A.2.2.2 Pattern-Based Sparse Attention

[9] Child, Rewon, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating Long Sequences with Sparse Transformers. arXiv preprint arXiv:1904.10509. <https://arxiv.org/abs/1904.10509>

Child et al. introduce Sparse Transformer, one of the foundational works on structured sparse attention. Instead of allowing every token to attend to every other token, the model uses structured patterns such as local and strided attention. Compared to Vaswani et al., this method sacrifices full connectivity in order to reduce computational cost. It also connects to later pattern-based methods because it shows that carefully designed sparse

layouts can still preserve useful long-range communication. Its limitation is that the patterns are fixed, which motivates later adaptive and hybrid approaches.

[10] Liu, Dong, and Yanxuan Yu. 2025. π -Attention: Periodic Sparse Transformers for Efficient Long-Context Modeling. arXiv preprint arXiv:2511.10696.
<https://arxiv.org/abs/2511.10696>

π -Attention introduces periodic sparse attention patterns for long-context modeling. It combines local attention with periodic long-range links, making it more expressive than purely local approaches. Compared to Sparse Transformer, it continues the idea of fixed structured patterns but focuses on periodic connectivity. Compared to adaptive methods like Routing Transformer, π -Attention is more predictable and easier to structure, but less flexible. This places it within pattern-based sparse attention, where efficiency comes from regular design rather than learned routing.

[11] Parmar, Niki, Ashish Vaswani, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2018. Image Transformer. arXiv preprint arXiv:1802.05751. <https://arxiv.org/abs/1802.05751>

Image Transformer applies self-attention to image generation and uses restricted attention to make computation more manageable. Compared to full self-attention in Vaswani et al., this work shows that structured attention is necessary when inputs become large, such as images. It connects to later vision-based sparse attention methods like Neighborhood Attention Transformer. Compared to NAT, Image Transformer is earlier and less specialized around local neighborhood attention, but it helps establish the need for efficient attention in vision. This paper supports the idea that sparse attention is important beyond natural language processing.

A.2.3 Hybrid Global–Local Attention

[12] Zaheer, Manzil, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big Bird: Transformers for Longer Sequences. arXiv preprint arXiv:2007.14062.
<https://arxiv.org/abs/2007.14062>

BigBird combines local, global, and random attention patterns to reduce the cost of processing long sequences. Compared to Longformer, BigBird adds random connections, which improve sequence-wide connectivity. This makes it more expressive than purely local sparse attention methods. Compared to Sparse Transformer, BigBird provides

stronger theoretical support for why sparse attention can preserve important properties of full attention. Its main trade-off is that the attention design becomes more complex because multiple connection types must be combined.

[13] Ainslie, Joshua, Santiago Ontanon, Chris Alberti, Vaclav Cvicek, Zachary Fisher, Philip Pham, Anirudh Ravula, Sumit Sanghai, Qifan Wang, and Li Yang. 2020. ETC: Encoding Long and Structured Inputs in Transformers. In Proceedings of EMNLP. <https://arxiv.org/abs/2004.08483>

ETC introduces a global-local attention structure for long and structured inputs. It separates tokens into local input tokens and global tokens that help summarize and share information across the sequence. Compared to Longformer, ETC more explicitly models structured inputs such as documents with hierarchy or organization. Compared to BigBird, it does not rely on random attention but instead uses a clearer global-local design. This makes ETC especially relevant for hybrid sparse attention, where the goal is to balance local efficiency with global information flow.

[14] Roy, Aurko, Mohammad Saffar, Ashish Vaswani, and David Grangier. 2021. Efficient Content-Based Sparse Attention with Routing Transformers. arXiv preprint arXiv:2003.05997. <https://arxiv.org/abs/2003.05997>

Routing Transformer uses content-based clustering to decide which tokens should attend to each other. Unlike Longformer and BigBird, which rely mostly on fixed sparse patterns, Routing Transformer makes attention more data-dependent. This gives the model more flexibility for sequences where important relationships are not strictly local. Compared to Adaptive Attention Span, it adapts by grouping similar tokens rather than learning attention distance. The trade-off is that routing adds extra complexity and may be harder to optimize than fixed sparse patterns.

A.2.4 Adaptive / Learnable Sparsity

[15] Sukhbaatar, Sainbayar, Edouard Grave, Piotr Bojanowski, and Armand Joulin. 2019. Adaptive Attention Span in Transformers. arXiv preprint arXiv:1905.07799. <https://arxiv.org/abs/1905.07799>

Adaptive Attention Span allows each attention head to learn how far it should attend. Compared to fixed sliding-window methods such as Longformer or SWAT, this gives the model more flexibility because the attention range is learned rather than manually chosen. Compared to Routing Transformer, it adapts attention by changing span length rather than

grouping tokens by content. This method helps address the limitation of fixed sparse patterns, which may not match every task or dataset. Its main trade-off is that adaptive sparsity can be more difficult to train and optimize efficiently.

[16] Liu, Zhuang, et al. 2021. Transformer Acceleration with Dynamic Sparse Attention. ArXiv preprint arXiv:2110.11299. <https://arxiv.org/abs/2110.11299>

Dynamic Sparse Attention focuses on making Transformer inference more efficient by dynamically selecting attention connections. Compared to fixed sparse methods such as Longformer or Sparse Transformer, this approach is more flexible because the attention pattern can change based on the input. It connects closely to adaptive methods like Routing Transformer and Adaptive Attention Span. However, dynamic attention can introduce extra overhead because the model must determine which connections to keep. This highlights the same central trade-off seen throughout the survey: flexibility improves expressiveness but can reduce simplicity and hardware efficiency.

A.2.5 Memory-Augmented Methods

[17] Dai, Zihang, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. 2019. Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. arXiv preprint arXiv:1901.02860. <https://arxiv.org/abs/1901.02860>

Transformer-XL introduces segment-level recurrence, allowing hidden states from previous segments to be reused as memory. Compared to standard Transformers, this extends context beyond a fixed input window. Unlike Longformer or BigBird, Transformer-XL does not primarily reduce cost by changing the attention pattern; instead, it improves long-range modeling through recurrence. This makes it a key memory-augmented alternative to sparse attention. It also connects to Compressive Transformer, which builds on this idea by storing older memory in compressed form.

[18] Rae, Jack W., Anna Potapenko, Siddhant M. Jayakumar, and Timothy Lillicrap. 2019. Compressive Transformers for Long-Range Sequence Modelling. arXiv preprint arXiv:1911.05507. <https://arxiv.org/abs/1911.05507>

Compressive Transformer extends Transformer-XL by adding compressed memory for older hidden states. This allows the model to retain longer-term information without storing every past representation at full size. Compared to Transformer-XL, it provides a more scalable memory mechanism for very long contexts. Compared to sparse attention methods, it solves long-context modeling by storing past information rather than limiting

attention connections. This paper shows how memory compression can complement sparse attention in future long-context architectures.

[19] Burtsev, Mikhail S., Yuri Kuratov, Anton Peganov, and Grigory V. Sapunov. 2021. Memory Transformer. arXiv preprint arXiv:2006.11527.

Memory Transformer introduces additional memory mechanisms to improve how Transformers retain information over long contexts. Unlike sparse attention methods, which reduce the number of token-to-token connections, memory-based approaches extend the model’s ability to store and reuse information. Compared to Transformer-XL, Memory Transformer explores more explicit memory structures. This connects to the broader memory-augmented category, where long-range dependency modeling is improved through storage rather than sparse connectivity. The trade-off is that memory mechanisms often add architectural and system complexity.

[20] Wu, Yuhuai, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. 2022. Memorizing Transformers. arXiv preprint arXiv:2203.08913.
<https://arxiv.org/abs/2203.08913>

Memorizing Transformers introduce an external memory system that allows models to retrieve information from a much larger history. Compared to Transformer-XL and Compressive Transformer, this approach relies more directly on retrieval from memory rather than only recurrence or compression. It provides stronger long-range access but can introduce retrieval overhead. Compared to sparse attention methods, it represents a different strategy for handling long context: extending memory rather than reducing attention links. This makes it important for the survey’s discussion of memory-augmented methods and future hybrid designs.

A.2.6 Complementary Efficiency Techniques

[21] Tay, Yi, Mostafa Dehghani, Dara Bahri, and Donald Metzler. 2020. Efficient Transformers: A Survey. arXiv preprint arXiv:2009.06732.
<https://arxiv.org/abs/2009.06732>

Tay et al. provide a broad survey of efficient Transformer methods, including sparse attention, low-rank methods, memory mechanisms, and other efficiency strategies. This paper is useful because it shows that sparse attention is only one part of a larger field focused on reducing Transformer cost. Compared to individual model papers like Longformer or BigBird, this survey provides a higher-level classification of approaches. It

helps support the need for organizing methods by design choices rather than listing them separately. This connects strongly to the taxonomy used in this survey project.

[22] Nawrot, Piotr, Robert Li, Renjie Huang, Sebastian Ruder, Kelly Marchisio, and Edoardo M. Ponti. 2026. The Sparse Frontier: Sparse Attention Trade-offs in Transformer LLMs. arXiv preprint arXiv:2504.17768. <https://arxiv.org/abs/2504.17768>

Nawrot et al. examine sparse attention trade-offs in modern large language models. This paper is important because it studies sparse attention in the context of current LLMs rather than only earlier Transformer models. Compared to Longformer or BigBird, which propose specific architectures, this work focuses more on evaluation and comparison across sparse methods. It supports the idea that no single sparse attention pattern is best for every task. This connects directly to the comparative discussion in the survey, especially the balance between efficiency, accuracy, and long-context reasoning.

[23] Dao, Tri, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv preprint arXiv:2205.14135. <https://arxiv.org/abs/2205.14135>

FlashAttention improves attention efficiency by optimizing how attention computation uses memory and hardware. Unlike Sparse Transformer, Longformer, or BigBird, it does not reduce attention connections or change the attention pattern. Instead, it keeps exact attention but makes it faster and more memory efficient through better IO-aware computation. This makes it a complementary method rather than a sparse attention method. It shows that Transformer efficiency can be improved both through algorithmic sparsity and through hardware-aware implementation.

[24] He, Bobby, and Thomas Hofmann. 2024. Simplifying Transformer Blocks. arXiv preprint arXiv:2311.01906. <https://arxiv.org/abs/2311.01906>

He and Hofmann study whether Transformer blocks can be simplified without losing strong performance. This paper is relevant because it addresses efficiency from the perspective of architectural simplification rather than sparse attention. Compared to FlashAttention, which optimizes computation at the kernel level, this work focuses on simplifying the model structure itself. Compared to Longformer or BigBird, it does not target long-context attention directly. However, it connects to the same broader goal of making Transformers more efficient and practical.

[25] Fedus, William, Barret Zoph, and Noam Shazeer. 2021. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. arXiv preprint arXiv:2101.03961. <https://arxiv.org/abs/2101.03961>

Switch Transformer introduces sparse computation through a Mixture-of-Experts design. Instead of sparsifying attention connections, it routes each token to only a small number of expert networks. Compared to Longformer or BigBird, this paper applies sparsity at the model level rather than the attention level. It connects to the broader theme that efficiency can come from activating only part of a large model. This makes it useful for the survey's discussion of current trends in sparse computation.

[26] Du, Nan, Yanping Huang, Andrew M. Dai, Zhifeng Chen, Orhan Firat, Noah Constant, Yonghui Wu, et al. 2022. GLaM: Efficient Scaling of Language Models with Mixture-of-Experts. arXiv preprint arXiv:2112.06905. <https://arxiv.org/abs/2112.06905>

GLaM is another large-scale Mixture-of-Experts model that improves efficiency by activating only selected parts of the model for each input. Like Switch Transformer, it uses sparsity in computation rather than sparsity in attention patterns. Compared to sparse attention methods, GLaM focuses on scaling model capacity while controlling computational cost. This connects to the survey because it shows that sparsity is a general strategy in Transformer design, not only an attention mechanism. Together, Switch Transformer and GLaM support the trend toward sparse computation in large models.